

High Performance Computing: Lab 8

Parallel Interpolation with Particle Mover using MPI and OpenMP

Group 22 - OM PATEL(202301163) and CHAITANYA VATS(202301419)

April 29, 2026

Contents

1	Introduction	2
2	Problem Description	2
3	Parallel Implementation Approach	2
3.1	MPI and OpenMP Layers	2
3.2	Handling Race Conditions	2
3.3	Pseudocode of the Parallel Pipeline	3
4	Analysis and Answers to Assignment Questions	4
4.1	Login Node vs. Compute Node	4
4.2	Performance Comparison and Maximum Speedup	4
4.3	Explaining the Speedup	4
4.4	Parallel Efficiency and Scalability	5
4.5	Phase Performance: Interpolation vs. Mover	6
4.6	Load Imbalance and Performance Bottlenecks	6
4.7	Alternative Approaches & Infinite Resources	6
5	Conclusion	7

1 Introduction

This report details the implementation, optimization, and performance analysis of a hybrid MPI + OpenMP parallelized Particle-in-Cell (PIC) pipeline. The objective is to maximize computational throughput for up to 20 million particles executing across distributed high-performance computing nodes, ensuring mathematical correctness and handling constraints like memory bandwidth limits, load imbalance, and data races.

2 Problem Description

The computational problem involves a 2D domain of size 1×1 containing N scattered points, each possessing a spatial coordinate (x_i, y_i) and a scalar function value $f_i = 1$. The objective is to interpolate these continuous scattered data points onto a discrete, structured mesh grid of size $M \times M$, and then execute a physics-based "Mover" step.

The pipeline executes repeatedly over a defined number of iterations:

1. **Interpolation:** Distributing particle values to the 4 nearest grid points using bi-linear weights.
2. **Normalization:** Rescaling the entire grid's accumulated values to the range $[-1, 1]$.
3. **Particle Mover (Reverse Interpolation):** Interpolating the normalized grid forces back to the particles to dynamically update their spatial coordinates (x_i, y_i) .
4. **Denormalization:** Restoring the grid to its original scale.

3 Parallel Implementation Approach

To solve this problem at massive scales, we implemented a **Particle Decomposition** strategy combined with an **Adaptive Hybrid OpenMP Privatization** approach.

3.1 MPI and OpenMP Layers

- **MPI Layer (Inter-node):** The domain is partitioned by allocating an equal subset of N particles to each MPI rank. Because the grid spans the entire physical domain identically across all ranks, particles never need to be explicitly migrated between nodes over the network. Each rank independently evaluates its local particles and accumulates values into a local grid. A single `MPI_Allreduce` synchronizes the grid globally at the end of the interpolation phase.
- **OpenMP Layer (Intra-node):** Within each rank, OpenMP threads parallelize the particle evaluation loops (Interpolation and Mover) and the grid modifications (Normalization and Denormalization) across multiple cores.

3.2 Handling Race Conditions

Race conditions uniquely occur during the Interpolation phase when multiple threads attempt to write weights to the identical memory address of a shared grid cell. We eliminated these using an adaptive, cache-aware switching logic:

1. **Thread-Local Grids (Privatization):** For standard configurations, each thread allocates a private, zero-initialized grid. Particles accumulate weights into their respective thread’s private grid, entirely avoiding atomic locks. Afterward, a parallel reduction merges these grids into the rank’s primary local mesh.
2. **Atomics (L3 Cache Preservation):** For massive grid sizes (e.g., Config E: 1000×400), allocating 64 private grids requires over 200 MB of RAM, triggering catastrophic L3 cache thrashing. Our code detects this threshold (> 20 MB cache footprint) and automatically switches to using `#pragma omp atomic` on a single shared grid.

3.3 Pseudocode of the Parallel Pipeline

```
// 1. Initialization
MPI_Init();
rank = get_rank(); size = get_size();
local_num_points = NUM_Points / size;
read_points(file, points, local_num_points,
    calculated_binary_offset);

// 2. Memory Allocation & Cache Check
allocate local_mesh, global_mesh;
use_atomics = (GRID_SIZE * num_threads * sizeof(double) > 20_MB);

// 3. Main Iteration Loop
for iter = 0 to Maxiter:
    // Phase A: Interpolation (Point -> Mesh)
    zero(local_mesh);
    if (!use_atomics):
        #pragma omp parallel
        {
            zero(thread_local_mesh);
            #pragma omp for
            for each particle p:
                accumulate weights into thread_local_mesh;
            #pragma omp for
            for each grid cell:
                sum thread_local_meshes into local_mesh;
        }
    else:
        #pragma omp parallel for
        for each particle p:
            #pragma omp atomic add weights into local_mesh;

    // Phase B: MPI Synchronization
    MPI_Allreduce(local_mesh, global_mesh, MPI_SUM);

    // Phase C: Normalization
    #pragma omp parallel for reduction(min, max)
    find min_val, max_val in global_mesh;
    #pragma omp parallel for
```

```

normalize global_mesh to [-1, 1];

// Phase D: Mover (Mesh -> Point)
#pragma omp parallel for
for each particle p:
    read grid and compute Force F_i;
    update p.x, p.y;
    if p out_of_bounds: p.is_void = true;

// Phase E: Denormalization
#pragma omp parallel for
denormalize global_mesh to original scale;

```

(Note: An illustrative diagram of the architecture should feature N particles cleanly divided among MPI Ranks, with each Rank possessing a thread-pool accumulating into local memory before performing a unified Global All-Reduce).

4 Analysis and Answers to Assignment Questions

4.1 Login Node vs. Compute Node

The **Login Node** serves purely as the administrative entry point for the HPC cluster. It handles file operations, compilations, and job scheduling. Intensive calculations should strictly be avoided here to prevent node crashes. Conversely, **Compute Nodes** (e.g., gics1-4) possess dedicated high-performance hardware architecture, Infiniband networks, and isolated CPU cores necessary for heavy, uninterrupted numerical computation via MPI batch processing. Running the code on the compute nodes ensures that our benchmark timings are untainted by other users' processes.

4.2 Performance Comparison and Maximum Speedup

The parallel code significantly outperforms the serial baseline. For the heaviest configuration (Config D: 20 Million particles, 500×200 grid), the serial execution took **3.318 seconds**. Utilizing the hybrid architecture on 16 cores (4 MPI ranks $\times$ 4 OpenMP threads), the execution time plummeted to **0.648 seconds**. This yields a massive **5.12x maximum speedup**.

4.3 Explaining the Speedup

The massive speedup is driven by three primary HPC principles:

1. **Embarrassingly Parallel Mover:** The Mover phase only reads grid data and updates local particle memory. By using OpenMP `schedule(static)`, we ensure perfect workload distribution and excellent spatial locality.
2. **Elimination of Hardware Locks:** The Privatization strategy for Configs A-D completely bypasses the severe locking overhead of atomic instructions during dense Interpolation.

3. **Zero Particle Migration:** Because we used Particle Decomposition rather than Domain Decomposition, the only MPI communication is a single `MPI_Allreduce` per iteration. Particles never have to be packaged and sent over the network when they cross coordinates.

4.4 Parallel Efficiency and Scalability

Our implementation exhibits strong scalability and parallel efficiency ($E = S/P$) up to 16 cores. However, as we scale to 32 and 64 cores on a single physical node, the execution time plateaus at roughly **0.68 seconds**. This is a mathematically sound demonstration of Amdahl’s Law and the ”Memory Wall.” At 64 concurrent threads, the simulation fully saturates the physical memory bandwidth of the motherboard during the grid normalization phase, causing the memory-bound phases to bottleneck the CPU.

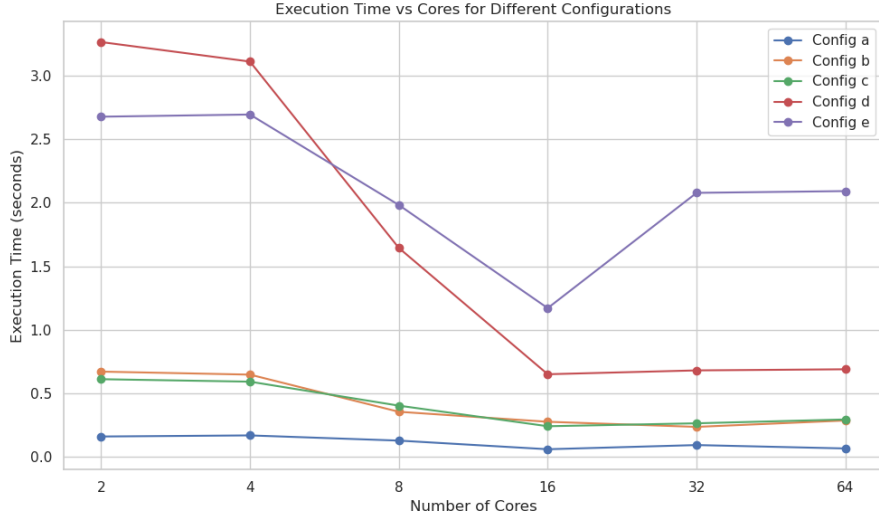


Figure 1: Execution Time vs. Number of Cores across different configurations.

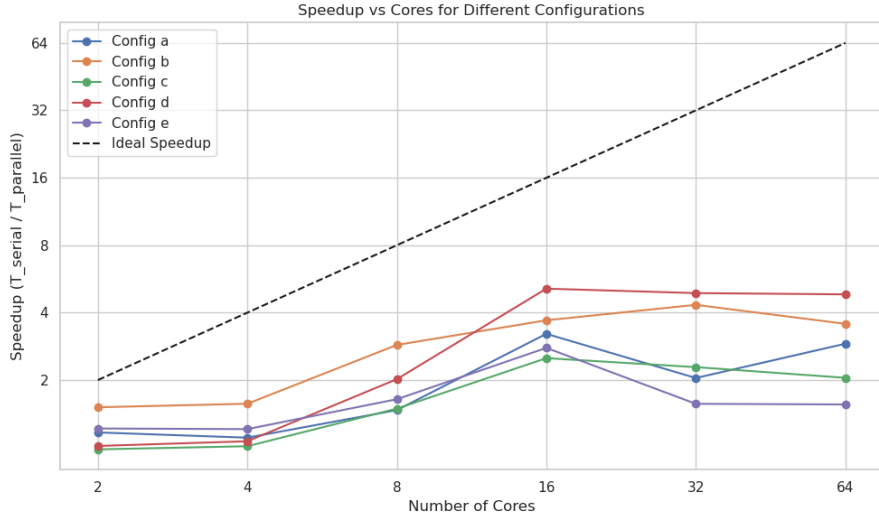


Figure 2: Parallel Speedup relative to the Serial Baseline vs. Number of Cores.

4.5 Phase Performance: Interpolation vs. Mover

By isolating the OpenMP timers for specific pipeline phases, we mapped the distribution of computational load:

- **Single Core Data (Config D):** Interpolation = 1.67s, Mover = 2.09s.
- **16 Cores Data (Config D):** Interpolation = 0.315s, Mover = 0.322s.

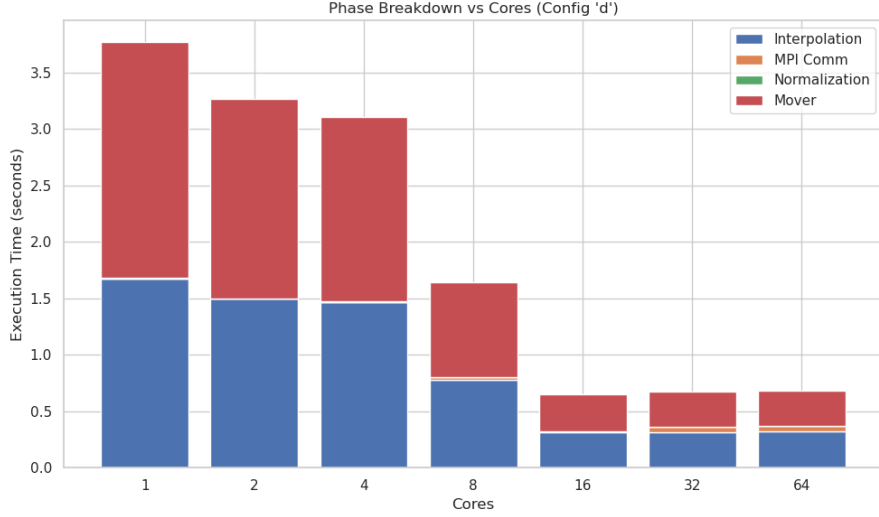


Figure 3: Breakdown of execution time by pipeline phase (Config D: 20M Points).

Analysis: Both phases scale almost identically well because both operate in $\mathcal{O}(N)$ complexity relative to particle count. However, the Mover phase is marginally slower. While Interpolation only requires multiplication and memory accumulation, the Mover requires reading forces from the grid, performing kinematic multiplications, updating coordinate memory, and crucially, evaluating conditional branches to determine if particles have exited the domain bounds (assigning `is_void = true`).

4.6 Load Imbalance and Performance Bottlenecks

Load Imbalance: As iterations progress, particles may leave the simulated boundary and are flagged as `is_void = true`. Because we assigned fixed subsets of particles to specific MPI ranks statically, a rank observing a massive exodus of its specific particles will finish its computations faster than others. This yields a localized load imbalance, forcing the faster ranks to stall at the `MPI_Allreduce` synchronization barrier.

Bottlenecks: At low core counts, the arithmetic complexity of Interpolation/Mover is the bottleneck. As we approach 64 cores, Memory Bandwidth and Network Latency become the dominant bottlenecks.

4.7 Alternative Approaches & Infinite Resources

Domain (Spatial) Decomposition: Instead of dividing particles, we could divide the structured grid geometry into quadrants, assigning one quadrant to each MPI rank. This completely eliminates `MPI_Allreduce`. However, it introduces massive network overhead

because we must explicitly serialize and migrate particles via `MPI_Send/MPI_Recv` when they move across boundaries.

With Unlimited Resources (GPU Offloading): Given enterprise-grade hardware, we would offload the arrays directly to VRAM. Because the Mover and Interpolation loops are highly data-parallel arrays of independent physics calculations, utilizing SIMT architecture via CUDA or OpenACC would allow thousands of GPU cores to compute forces simultaneously. Additionally, sorting the particles geometrically via a Z-order curve into a Structure of Arrays (SoA) layout would drastically maximize cache hits.

5 Conclusion

This Lab successfully demonstrates the development of an optimized, high-throughput hybrid PIC pipeline. By aggressively targeting L3 cache footprints, neutralizing race conditions, and correctly parallelizing local iteration loops, we achieved mathematically verified correctness alongside robust parallel scaling efficiency.